

Universidad de Antioquia
Facultad de Ingeniería

Informática II – Semestre 2026-2

Desafío I

Implementación de Sweet Crush mediante manipulación de bits en C++

Informe preliminar

Juan Martín López González
Juan Camilo David Zapata

11 de Septiembre de 2026

1. Contextualización

El presente informe documenta el análisis y diseño preliminar para la implementación del juego Sweet Crush. Este desafío pone a prueba a los estudiantes para verificar que han adquirido las destrezas y conocimientos fundamentales de la programación en C++, enfocándose principalmente en la gestión de memoria dinámica, el uso de apuntadores y la manipulación de datos a nivel de bits. El reto reside en administrar adecuadamente las operaciones del juego garantizando un uso eficiente de la memoria RAM.

2. Análisis del problema

2.1. Decisiones de análisis derivadas y su justificación

Para almacenar el tablero en la memoria principal, se ha tomado la decisión de utilizar un arreglo dinámico de tipo `unsigned char*`.

Esta opción es la más prudente porque garantiza un control atómico de exactamente 8 bits por bloque. El uso explícito de `unsigned` es muy importante: evita un mal comportamiento del compilador al aplicar desplazamientos de bits hacia la derecha. En variables con signo, un desplazamiento puede arrastrar el bit negativo llenando el espacio con (1s), lo cual corrompería las fichas adyacentes. Se prefirió `char` sobre enteros más grandes para mantener el seguimiento de las fronteras de bits durante esta etapa de diseño.

3. Diseño general de la solución

3.1. Arquitectura de módulos

La solución estará administrada de la siguiente manera usando multiarchivos:

- `main.cpp`: Manejará el bucle de ejecución, alojará el estado global y el menú.
- `memoria.h /.cpp`: Encargado de la asignación de memoria, liberación y comprobación del 65 %.
- `tablero.h /.cpp`: Contendrá la lógica a bajo nivel de máscaras y empaquetado.
- `logica.h /.cpp`: Ejecutará la verificaciones y reglas (cascadas y combinaciones).

3.2. Diseño de las Funciones Principales

Hemos identificado para la solución de este desafío las siguientes funciones principales (No únicas):

3.2.1. 1. Creación y asignación exacta de memoria

Calcula los bytes necesarios a partir de la fórmula $(\text{Filas})(\text{Columnas})3$, dividiéndolo entre 8 (aplicando la función techo) y reservando memoria dinámica.

Se decidió implementar este cálculo añadiendo un byte adicional de seguridad al final de la asignación. Esto es muy importante porque al extraer una ficha ubicada en el último índice, la lectura de sus 3 bits podría cruzar la frontera del byte válido e intentar acceder a memoria no reservada. Ese byte extra actúa como un margen para evitar violaciones de segmento.

3.2.2. 2. Generador de aleatoriedad uniforme

Devuelve un patrón de 3 bits para uno de los seis tipos de fichas.

Aislamos esta tarea matemática para garantizar la probabilidad uniforme exigida. Si el tablero se satura de un solo tipo de ficha, las mecánicas fallarán. Tenerla en una función independiente nos permite ejecutar pruebas previas para comprobar que la probabilidad de cada ficha sea exactamente la correspondiente para el número de fichas necesarias.

3.2.3. 3. Empaquetador a nivel de bits

Traduce una coordenada lógica (fila, columna) a un índice unidimensional e inserta la ficha modificando la memoria.

Esta es la decisión central. Decidimos construir esta interfaz intermedia para que el resto del programa opere sobre una matriz. Manejar explícitamente las particiones de memoria directamente dentro de los bucles de detección haría el código inmanejable.

3.2.4. 4. Motor de detección y cascada

Escanea las combinaciones, las marca, desplaza las fichas restantes hacia abajo y rellena los espacios vacíos. Optamos por un diseño de dos fases: primero marcar las fichas a eliminar y luego ejecutar la eliminación. Adicionalmente, el efecto en cadena se gestionará iterativamente en lugar de usar recursividad, evitando desbordamientos si ocurren muchas cascadas seguidas.